Кодирование категориальных признаков: expanding mean и хэширование

Елена Кантонистова

ПЛАН ЛЕКЦИИ

- Напоминание: счётчики и их опасность
- Счётчики по схеме expanding mean
- Хэширование признаков
- Где это используют на практике

НАПОМИНАНИЕ: СЧЁТЧИКИ

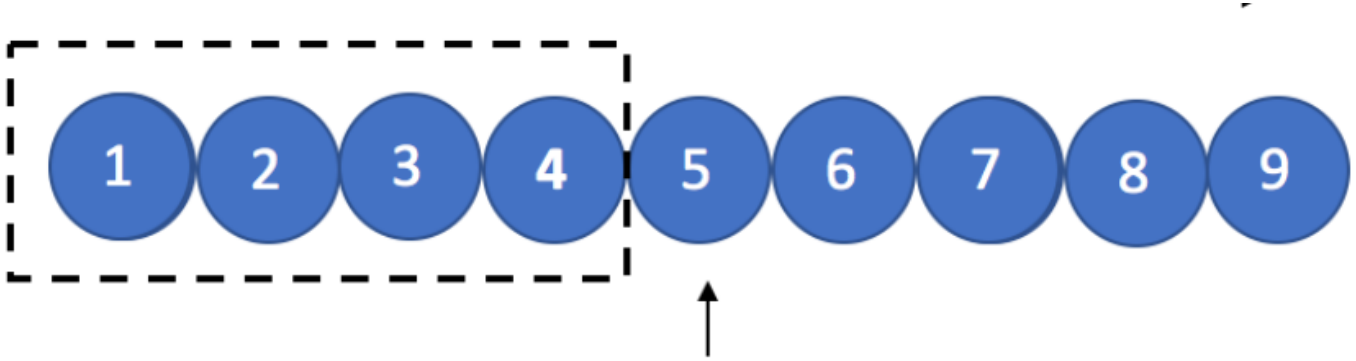
- Счётчик (mean target encoding) заменяет категорию на среднее значение таргета в ней:

$$\mathrm{code(c)} = \mathrm{mean(target} \mid \mathrm{f(x)} = \mathrm{c)}$$

- Опасность: в признак попадает информация о целевой переменной, в том числе о таргете **того же объекта** → модель переобучается
- Уже знаем два способа борьбы:
  - сглаживание к глобальному среднему
  - вычисление счётчиков на кросс-валидации
- Третий способ — **expanding mean**

EXPANDING MEAN: ИДЕЯ

Суть схемы заключается в том, чтобы пройти по отсортированному в определенном порядке датасету и для подсчета счетчика для строки *m* использовать строки от *0* до *m-1*.



Running mean calculation.
Numbers are assigned randomly to each observation. Only 1-4 are used to find encoding for 5

EXPANDING MEAN: ФОРМУЛА

- Упорядочим объекты (по времени или случайно). Для объекта с номером *m* и категорией *c*:

$$\text{code}_m = \frac{\sum_{i<m,\ c_i=c} y_i + \alpha \cdot \text{prior}}{n_{<m}(c) + \alpha}$$

- $n_{<m}(c)$ — сколько раз категория *c* встретилась **до** объекта *m*
- prior — глобальное среднее таргета, α — сила сглаживания
- Таргет самого объекта *m* в его код не попадает → нет утечки
- На тесте используем статистики по **всей** обучающей выборке

EXPANDING MEAN: ПРИМЕР

prior = 5/8 = 0.625, α = 1. Объекты идут в порядке времени.

№	Город	y	Σ y до m	n до m	code
1	A	1	0	0	0.625
2	B	0	0	0	0.625
3	A	0	1	1	0.812
4	A	1	1	2	0.542
5	B	1	0	1	0.312
6	B	0	1	2	0.542
7	A	1	2	3	0.656
8	B	1	1	3	0.406

Например, объект 4: (1 + 1·0.625) / (2 + 1) ≈ 0.542. Первые объекты каждой категории получают код, близкий к prior.

EXPANDING MEAN: ОСОБЕННОСТИ

- **+** Объект не «видит» свой таргет → счётчик честный
- **+** Считается за один проход по данным
- **–** У первых объектов мало истории → код шумный. Помогают сглаживание и **несколько случайных перестановок** с усреднением
- **–** Результат зависит от порядка объектов:
  - данные со временем — упорядочиваем по времени, это ещё и честно имитирует прод
  - данные без времени — случайная перестановка

EXPANDING MEAN В КОДЕ

Вручную на pandas:

```
df = df.sort_values('time')
prior, alpha = df['target'].mean(), 1

grp = df.groupby('city')['target']
sum_before = grp.cumsum() - df['target']   # сумма таргета до объекта
cnt_before = grp.cumcount()                # число объектов до него

df['city_te'] = (sum_before + alpha * prior) / (cnt_before + alpha)
```

Готовые реализации:

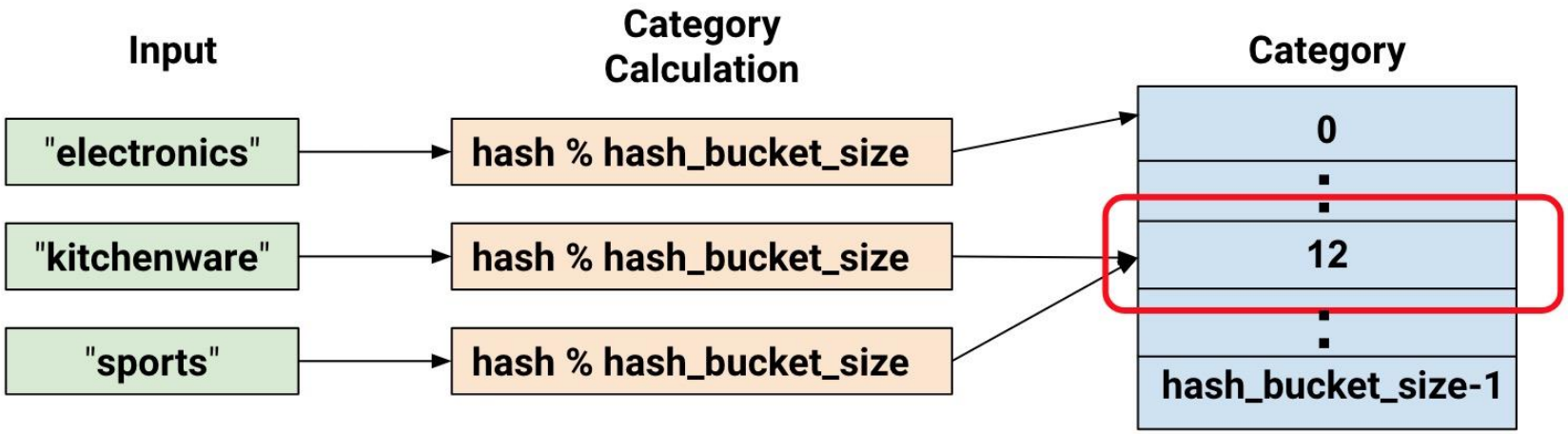
- `CatBoostEncoder` из библиотеки category_encoders
- CatBoost делает это сам для всех `cat_features`

ХЭШИРОВАНИЕ ПРИЗНАКОВ

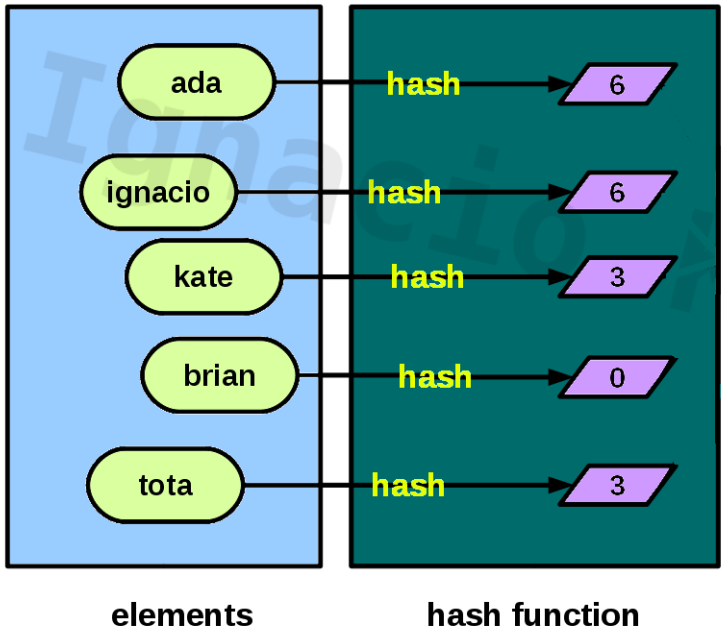
- Если у категориального признака слишком много значений, скажем, миллион, то после применения one-hot кодировки мы получим миллион новых столбцов. С такой огромной матрицей тяжело работать.
- Хэширование развивает идею one-hot кодирования, но позволяет получать любое заранее заданное число новых числовых столбцов после кодировки.

АЛГОРИТМ ХЭШИРОВАНИЯ

1) Для каждого значения признака вычисляем значение некоторой функции – хэш-функции (hash)
2) Задаем hash_bucket_size – итоговое количество различных значений категориального признака.
3) Берем остаток: hash % hash_bucket_size – тем самым кодируем каждое значение признака числом от 0 до hash_bucket_size-1.
4) Дальше к полученным числам применяем OHE.



ХЭШИРОВАНИЕ ПРИЗНАКОВ: ПРИМЕР



0	1	2	3	4	5	6
0	0	0	0	0	0	1
0	0	0	0	0	0	1
0	0	0	1	0	0	0
1	0	0	0	0	0	0
0	0	0	1	0	0	0

ЧТО ДЕЛАЕТ ХЭШ-ФУНКЦИЯ

- Хэш-функция раскладывает значения по корзинам **псевдослучайно и равномерно**. О частоте значений она ничего не знает
- Разные значения могут попасть в одну корзину — это **коллизия**. Модель их больше не различает
- Чем больше корзин, тем меньше коллизий:
  - 10 000 значений, 2^{14} ≈ 16 тыс. корзин → коллизия у ~45% значений
  - 10 000 значений, 2^{20} ≈ 1 млн корзин → коллизия у ~1% значений
- Коллизии редких значений почти не вредят: они мало влияют на модель
- Знаковое хэширование (+1 / −1 по второй хэш-функции) уменьшает искажения от коллизий

ХЭШИРОВАНИЕ В КОДЕ

```
from sklearn.feature_extraction import FeatureHasher

hasher = FeatureHasher(n_features=2**18,      # число корзин
                       input_type='string',
                       alternate_sign=True)   # знаковое хэширование

X_hashed = hasher.transform([[f'city={c}', f'device={d}']
                             for c, d in zip(df['city'], df['device'])])
# X_hashed — разреженная матрица n × 2^18
```

- Словарь категорий хранить не нужно: `fit` ничего не запоминает
- Для текстов есть `HashingVectorizer` — bag-of-words без словаря

ХЭШИРОВАНИЕ: ПЛЮСЫ И МИНУСЫ

Плюсы

- Размер признакового пространства задаём сами
- Кодирует любое значение, **в том числе новое**, которого не было в трейне
- Не нужно хранить словарь — удобно для потоков и онлайн-обучения
- Легко кодировать пересечения признаков

Минусы

- Коллизии: разные значения сливаются
- Признаки неинтерпретируемы: по корзине не понять, что в ней лежит
- При небольшом числе категорий выигрыша нет — проще one-hot

КЕЙСЫ: EXPANDING MEAN

CatBoost (Яндекс)

«Ordered target statistics» — это expanding mean по случайной перестановке. CatBoost считает их по умолчанию для всех категориальных признаков и усредняет по нескольким перестановкам.

Антифрод и скоринг на транзакциях

Доля мошеннических операций у магазина или устройства, посчитанная только по *прошлым* транзакциям. Признак не заглядывает в будущее, и в проде его легко считать онлайн.

Реклама и рекомендации по логам событий

Историческая конверсия объявления, товара или пользователя на момент показа — без утечки будущих кликов в обучение.

КЕЙСЫ: ХЭШИРОВАНИЕ

Персонализированный спам-фильтр

Weinberger и др. (2009): признаки «пользователь × слово» для сотен тысяч пользователей хэшируются в один вектор фиксированной длины — одна модель вместо отдельной на каждого.

Предсказание кликов (CTR) в рекламе

Миллионы ID объявлений, сайтов и пользователей плюс их пересечения. Vowpal Wabbit хэширует все признаки по умолчанию (2^{18} корзин).

Тексты и рекомендации

`HashingVectorizer` для огромных корпусов и потоков текста; хэш-корзины для эмбеддингов ID в нейросетевых рекомендательных моделях.

ЧТО ПОЧИТАТЬ

- Prokhorenkova et al. CatBoost: unbiased boosting with categorical features. arxiv.org/abs/1706.09516
- Weinberger et al. Feature Hashing for Large Scale Multitask Learning. arxiv.org/abs/0902.2206
- Статья про хэширование: arxiv.org/abs/1509.05472
- Habr (ODS): habr.com/ru/company/ods/blog/326418/